



UNIVERSITÀ DEGLI STUDI DI GENOVA

DIBRIS

DEPARTMENT OF INFORMATICS,
BIOENGINEERING AND ROBOTICS ENGINEERING

MODELLING AND CONTROL OF MANIPULATORS

Exam session: 16/01/2026

Professors:

Enrico Simetti
Giorgio Cannata

Tutors:

Luca Tarasi
Simone Borelli

January 16, 2026

Contents

1	Exam description	3
2	Exercise	3

Mathematical expression	Definition	MATLAB expression
$\langle w \rangle$	World Coordinate Frame	w
${}^a_b R$	Rotation matrix of frame $\langle b \rangle$ with respect to frame $\langle a \rangle$	aRb
${}^a_b T$	Transformation matrix of frame $\langle b \rangle$ with respect to frame $\langle a \rangle$	aTb
${}^a O_b$	Vector defining frame $\langle b \rangle$ with respect to frame $\langle a \rangle$	aOb
${}^b \eta_g$	Vector defining the orientation of frame $\langle b \rangle$ with respect to frame $\langle a \rangle$ given in terms of Yaw-Pitch-Roll angles	b_eta_g
${}^b v_{g/b}$	Velocity of frame $\langle g \rangle$ with respect to frame $\langle b \rangle$	bVg

Table 1: Nomenclature Table

1 Exam description

The Modeling and Control of Manipulator exam consists of a single exercise that summarizes all the topics discussed during the lectures. It should be solved using MATLAB, using the tools implemented during the practical lessons.

We have prepared a compressed folder in which you can find the code template, which consists of a *main.m* and a folder */include*. The main file should be compiled following the template. If it is completed correctly, the code can be launched and should output the results of the exam. The */include* folder contains the template of the MATLAB classes implemented during the practical lessons:

- *geometricModel*: which implements methods to calculate the direct geometry of the manipulator.
- *kinematicModel*: which implements methods to calculate the Jacobian matrix of the manipulator.
- *kinematicSimulation*: it simulates the kinematic of the robot.
- *cartesianControl*: which implements methods for the control of the manipulator.

You can add additional functions to the */include* folder. You are not bound to follow a standard template for the functions. It is strictly forbidden to use built-in MATLAB functions instead of implementing the methods we have seen during the practical lessons. In the main script and the given function templates, the lines of code to be modified are usually distinguished by three dots ...

The function in the subfolder */include/utills* are already implemented and ready to use, therefore they must not be modified. **Please remember, when it is necessary, to use the functions that you have already developed in your previous assignment implementations. Re-implementation of the body of such functions in the main script, even if your code works, will result in a loss of points.**

2 Exercise

Given the geometric model of the industrial manipulator in Figure 1, you have to implement an inverse kinematics loop to control the manipulator. The task will have two phases: in the first phase, you will control the tool of the manipulator, identified by the frame $\langle t \rangle$. In the second phase, you will control the end-effector, associated to frame $\langle e \rangle$. Consider that the tool frame is rigidly attached to the robot end-effector, and the joint 6 is prismatic, while the others are rotational. The initial configuration of the robot must be set to $q = [\pi/2, -\pi/4, 0, -\pi/4, 0, 0.15, \pi/4]$. Follow the steps:

Q1 Compute the Cartesian error between the robot tool frame b_tT and the goal frame ${}^b_{g,tool}T$ (called *bTg_tool* in the code). The orientation of the goal is given by the YPR triplet $\Gamma_{g,tool} = [1.0363, 0.3218, -0.0077]$ rad, while its translation is ${}^bO_{g,tool} = [0.0087, 0.5808, 0.6714]^T$ m.

Q2 Compute the desired angular and linear reference velocities of the end-effector with respect to the base.

Q3 Compute the desired joint velocities $\dot{q}$

Q4 Simulate the robot motion integrating the joint velocities in time. We expect to see the manipulator moving the tool frame, reaching the goal frame.

Q5 Now repeat the steps from Q1 to Q4, this time controlling the end-effector (ee) to reach the goal frame ${}^b_{g,ee}T$ (called *bTg_ee* in the code). The orientation of the goal is given by the YPR triplet $\Gamma_{g,ee} = [-0.3876, -0.3614, -0.3647]$ rad, while its translation is ${}^bO_{g,ee} = [0.0202, 0.7779, 0.9927]^T$ m.

Note 1: you must handle the change of tool control point from the first phase to the second phase.

Note 2: to implement the two phases, do not duplicate the code of the control loop, in which case there will be a penalty, even if the code works.

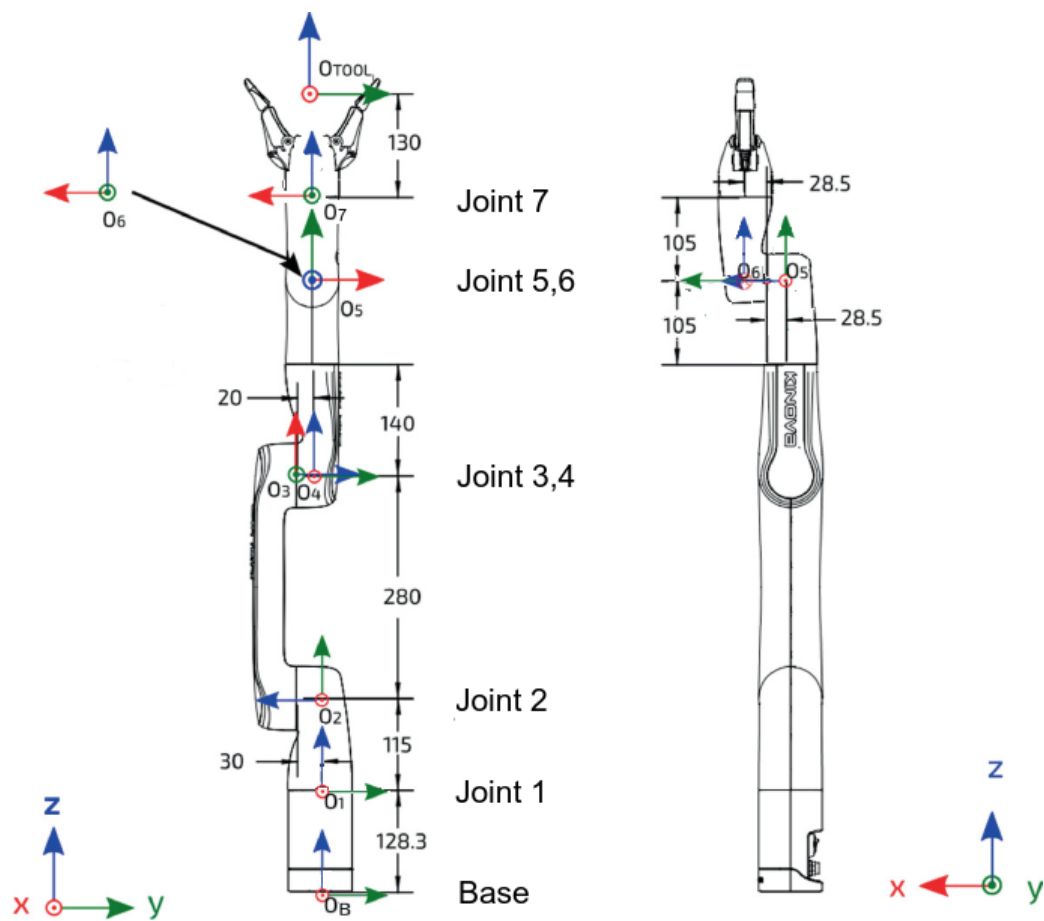


Figure 1: CAD model of the robot.